Lenguaje de Programacion

Es un conjunto de reglas sintacticas, semanticas y simbolos formales que permite a un ser humano escribir instrucciones estructuradas para que una computadora las interprete, procese y ejecute

Elementos Esenciales de un lenguaje de programacion:

Sintaxis: Es el conjunto de reglas gramaticales que dictan como deben escribirse las palabras, simbolos y estructuras para que el codigo sea valido.

Semantica: Es el significado o la accion logica que realiza cada instruccion escrita

Vocabulario / Palabra Reservada: Palabra clave con significado especial para el lenguaje que no pueden usarse para otros fines. *Ejemplo*: while, return, etc.

Programacion: Es el proceso de diseñar, escribir, probar y mantener codigo fuente para dar instrucciones a una computadora permitiendo ejecutar tareas especificas o resolver problemas de forma automatica.

Algoritmo: Es una secuencia finita, ordenada y logica de pasos o instrucciones precisos que deben seguirse para realizar una tarea o resolver un problema concreto.

Lenguaje de Alto Nivel

Es un lenguaje diseñado para ser leido y escrito facilmente por los seres humanos. Utiliza palabras en ingles y estructuras logicas cercanas al lenguaje natural, independizando al programador de la orquitectura interna de hardware. *Ejemplo*: C, C++, Python, Java Script.

Lenguaje de Bajo Nivel

Es un lenguaje que opera con minima o nula abstraccion del hardware, diseñado para comunicarse e interactuar directamente con la memoria y el procesador de la computadora. *Ejemplo*: lenguaje maquina y lenguaje ensambleador.

Entorno de Desarrollo Integrado (IDE)

Es una aplicacion de software que reune todas las herramientas necesarias para que un desarrollador escriba y pruebe codigo en un solo lugar.

Componentes principales de un IDE

Editor de codigo fuente: Incluye resaltado de sintaxis autocompletado y deteccion de errores en tiempo real.

Compilador / Interpretador Integrado: Permite ejecutar el programa con un solo click.

Depurador (Debugger): Herramienta para inspeccionar la memoria, avanzar linea por linea y rastrear errores

Gestor de proyectos: Organiza multiples archivos, bibliotecas y configuraciones del programa

Variable: Una variable es un espacio reservado en la memoria RAM del ordenador que se utiliza para almacenar un valor que puede cambiar a lo largo de la ejecución del programa. Tiene un nombre (Identificador) y un tipo de dato asociado. **Ejemplo:**

- Num1 = 5
  base = 10
- Convertir grados celsius a grados Fahrenheit
  °F = (°C × 18) + 32

Constante: Una constante es un valor almacenado en memoria que no puede cambiar ni ser modificada una vez que a sido definido durante la ejecución del programa. Se utiliza para evitar errores y representar valores fijos del dominio.

Tipos de datos:

Categoría	Tipo de dato	Tamaño Hábitual	Rango
entero	int	4 bytes	-2 147 483 648 a 2 147 483 648
	short	2 bytes	-32 768 a 32 768
	long	4 o 8 bytes	-2 147 483 648 a 9 223 372 036 854 775
	long long	8 bytes	-9 223 372 036 854 775 808 a 9 223 372 036 854 775 807
Punto flotante	float	4 bytes	$\pm 3.4 \times 10^{-38}$ a $\pm 3.4 \times 10^{38}$
	double	8 bytes	$\pm 1.7 \times 10^{-308}$ a $\pm 1.7 \times 10^{308}$
Caracteres	char	1 bytes	-128 a 127
	wchar_t	2 o 4 bytes	—
Lógicos	bool	1 bytes	0 a 1

DFD

Es una representacion grafica que muestra como fluye la informacion a traves de un sistema o proceso

Elementos Principales :

Simbolo	Nombre	Funcion
Cuadro / Rectangulo	Entidad Externa	Representa la fuente u origen de datos
Circulo / Ovalo	Proceso	Operacion o Funcion que transforma los datos
Lineas horizontales / rectangulo abierto	Almacen de datos	Lugar donde se guardan los datos temporal o permanente
Flecha	Flujo de datos	Muestra la direcciones en lo que viaja la informacion entre los elementos

Pseint

Es una herramienta educativa de software libre diseñado para ayudar a estudiantes a dar sus primeros pasos en la programacion y el desarrollo de la logica algoritmica

Utiliza pseudocodigo en español lo que permite centrarse en los conceptos fundamentales de la programacion (variables, condicionales, bucles y funciones) sin la complicidad de la sintaxis estricta de un lenguaje real

Conversion de tiempo (Segundo a minutos/horas): Leer un valor en segundos e imprime a cuantos minutos y horas equivale

Calcula IGV de un producto: Solicita el precio base de un producto y calcula el monto del IGV junto con el precio total final

Area y perimetro de un rectangulo: Pide la longitud de la base y la altura de un rectangulo para imprimir su area y perimetro

Switch - case

En C++, switch/case, es una estructura de control de seleccion multiple que evalua el valor de una variable y ejecuta un bloque de codigo segun la coincidencia con una lista de casos patros o caracteres.

Ejemplo:

```
switch (variable) {
    case 1 (variable 1)
        //ejecuta un codigo ;
            break;
    case 2 (variable 2)
        //ejecuta un codigo;
            break;
    default:
        codigo no coincide con ningun previo case;
        break; }
```

Operadores

Operadores Aritmeticos

+ , - , * , / , % , + + , - -

Operadores Relacionales

== , != , > , < , >= , <=

Operadores Logicos

&& , || , ! , ^ , ~

If - Else : Las estructuras if - else son sentencias de control condicionales que permite tomar decisiones en el codigo ejecutando bloques de instruccion unicamente cuando se cumple una condicion

Estructura If (condicional simple): Evalua una condicion, si la condicion resulta verdadero (true), se ejecuta el codigo dentro del bloque. Si es falsa (false), el programa emite ese bloque y continua con la siguiente instruccion

Estructura If - Else (condicional doble): Permite definir en camino alternativa, si la condicion resulta verdadero (true) se ejecuta el primer bloque. Si es falso (false), se ejecuta obligatoriamente el bloque dentro del else

Estructura Repetitiva

También llamada bucle ciclo o iteración es un bloque de código que permite ejecutar una o varias instrucciones de forma recurrente mientras se cumpla una determinada condición lógica

Su propósito principal es automatizar tareas repetitivas, evitar la duplicidad de código y procesar colecciones de datos (como listas o arreglos) de manera diferente

```
for ( inicio ; final ; incremento )
for (i = 1 ; i <= 10 ; i ++ )
```

Iteración: Una iteración es cada una de las repeticiones o ejecuciones individuales de un bucle. Si un bucle se ejecuta S veces, se dice que se realiza S iteraciones

Contador: Es una variable entera cuyo valor se incrementa o decrementa en una cantidad constante (generalmente de 1 en 1) en cada iteración. Se utiliza para registrar la cantidad de veces que ocurre un evento o se repite un ciclo

```
contador = contador +1;
contador por = contador por + 1;
contador - por = contador - por +1;
```

Acumulador: Es una variable que almacena la suma o multiplicación sucesiva de valores variables en cada iteración. Se utiliza para obtener totales, promedios, o sumatorias acumulados.

Control de Flujo Iterativo

C++ proporciona sentencias especiales para llevar la ejecución normal de una estructura repetitiva

Break: Interrumpe y sale inmediatamente del bucle, ignorando las iteraciones restantes

Continue: Salta el resto de las instrucciones de la iteración actual y pasa directamente a la siguiente iteración (evaluando de nuevo la condición de incremento)

Estructura Repetitiva While (mientras)

Es un bucle o ciclo de control que ejecuta un bloque de codigo de forma iterativa mientras se cumpla una condicion logica determinada

A diferencia de otros bucles como (for) el bucle while se utiliza principalmente cuando no se conoce con exactitud el numero exacto de iteracion que se van a realizar antes de iniciar la ejecucion

Funcionamiento Basico :

1= Evaluacion : Antes de ejecutar el codigo se evalua la condicion logica

2= Ejecucion : Si la condicion es verdadero (true), se ejecuta el cuerpo del bucle

3= Repeticion : Al terminar el bloque, se vuelve a evaluar la condicion el proceso continua hasta que la condicion pase a ser falsa (false)

4= Finalizacion : Si la condicion es falsa desde el inicio, el bloque dentro del bucle no se ejecuta ni una sola vez

Estructura Repetitiva Do - While

Los estructuras repetitivas do - while (hacer - mientras) es un bucle o ciclo de control que ejecuta un bloque de codigo al menos una sola vez y luego continua repitiendolo mientras se cumpla una condicion logica determinada

Funcionamiento Basico :

1= Ejecucion Obligatoria : El programa entra al ciclo y ejecuta las instrucciones del bloque directamente, sin comprobar ninguna condicion previa

2= Evaluacion Posterior : Al llegar al final del bloque, se evalua la condicion logica expresada en el while

3= Repeticion : Si la condicion es verdadero (true), el flujo regresa al inicio del bloque para repetir el proceso

4= Finalizacion : Si la condicion es falsa (false), el ciclo termina y el programa continua con la instruccion siguiente

Arreglos

Un arreglo (también conocido como array, vector o matriz) es una estructura de datos homogenea y de tamaño R que permite almacenar una colección de elementos del mismo tipo bajo un único nombre, cada elemento individual dentro del arreglo se identifica y se accede mediante 1 o mas índices numéricos

Arreglo Unidimensional (vector)

Un arreglo unidimensional es una colección lineal de datos que se organizan de forma secuencial, uno al lado del otro en la memoria del computador. Se puede visualizar como una sola fila con multiples casillas.

Ejemplo

0	1	2	3	4
2	14	15	18	20

Tipo de dato: int
int notas [5];

Arreglo Bidimensional (matriz)

Un arreglo bidimensional es una estructura organizada en una tabla de dos dimensiones, filas y columnas. Se puede imaginar como una cuadricula o una hoja de calculo, donde la intersección de una fila y una columna determina una posición o casillo específica

Ejemplo:

$$\begin{bmatrix} 14 & 15 & 16 \\ 8 & 13 & 20 \end{bmatrix}$$

int matriz [2][3];